

SITUAÇÃO-PROBLEMA INTEGRADORA - “Cruzamento 4.0

Viviane Vitoria

Vitoria Nunes

Sumário

1. Levantamento de Requisitos (Fase 1)	3
2. Sistema Operacional e Segurança (Fase 3)	4
3. Algoritmo do Semáforo Inteligente (Fase 4)	5
4. Versionamento (Fase 5)	5
CODIGOS PRONTO	6
PÁGINA WEB SEMÁFORO	11
ORGANIZAÇÃO GITHUB	12
FLUXOGRAMA E DIAGRAMA	13
BIOGRAFIA:	15

Documentação do Projeto: Cruzamento 4.0

1. Levantamento de Requisitos (Fase 1)

Com base na situação-problema identificada pela prefeitura, foram definidos os seguintes parâmetros:

Requisitos Funcionais (RF)

- **RF01:** O sistema deve alternar os estados das luzes automaticamente (Via A e Via B).
- **RF02:** O sistema deve detectar e reagir a condições de chuva forte.
- **RF03:** O sistema deve priorizar a via com fluxo de veículos superior a 70%.
- **RF04:** Deve acionar modo de segurança (amarelo piscante) em caso de falha de sensores.
- **RF05:** Deve manter a operação local caso o servidor central fique offline.
- **RF06:** Registro de log em tempo real para auditoria da CET.
- **RF07:** Interface visual para monitoramento do status dos sensores.
- **RF08:** Botão de reset para normalização do sistema após intervenção.

Requisitos Não Funcionais (RNF)

- **RNF01:** Tempo de resposta para transição de estados inferior a 1 segundo.
- **RNF02:** Interface responsiva compatível com dispositivos de monitoramento.
- **RNF03:** Comunicação segura via protocolo MQTT ou TCP/IP.
- **RNF04:** Código modular seguindo práticas de *Clean Code*.
- **RNF05:** Alta disponibilidade (o sistema não pode parar durante a queda do servidor).
- **RNF06:** Versionamento completo via Git.

Histórias de Usuário

1. **Como Operador da CET**, quero visualizar o fluxo em tempo real para antecipar congestionamentos.
2. **Como Motorista**, quero que o semáforo entre em modo de atenção durante chuvas para evitar acidentes.
3. **Como Técnico de Manutenção**, quero ser avisado por logs quando um sensor falhar.

2. Sistema Operacional e Segurança (Fase 3)

A escolha sugerida para o servidor local é o **Ubuntu Server**:

Item	Ubuntu Server	Justificativa
Custo	Zero (Open Source)	Redução de custos públicos.
Segurança	Alta (Permissões granulares)	Proteção contra falhas identificadas.
Suporte IoT	Nativo (Docker/MQTT)	Facilidade de integração com ESP32.

Políticas de Segurança (PSI):

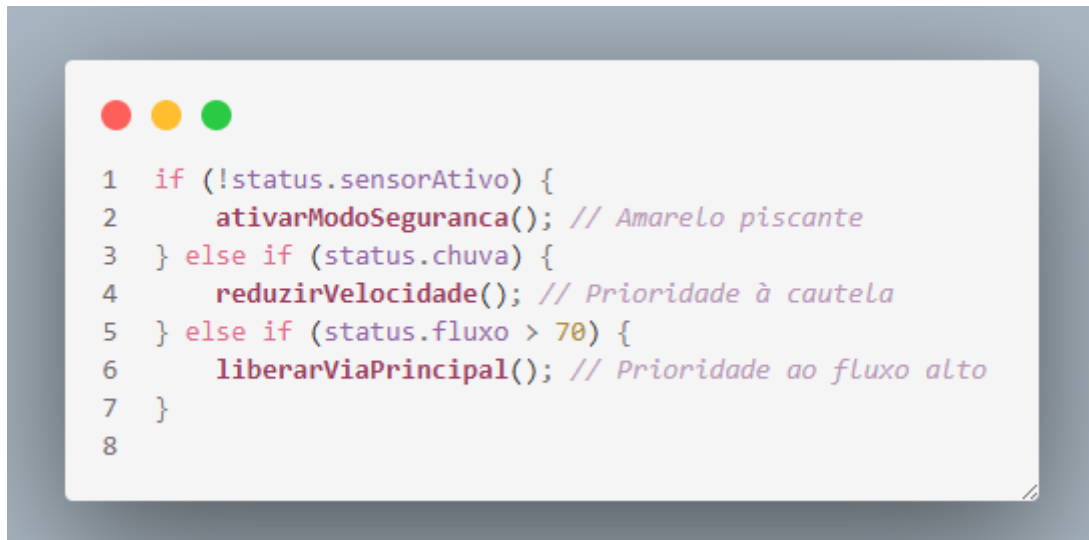
- **Acessos:** Bloqueio de portas desnecessárias via Firewall, permitindo apenas porta 1883 (MQTT).
- **Senhas:** Política de troca trimestral para acesso aos painéis de controle.
- **Backup:** Backup diário automático dos logs de tráfego para o servidor secundário.

3. Algoritmo do Semáforo Inteligente (Fase 4)

O código desenvolvido utiliza estruturas condicionais para gerenciar os estados críticos:

Javascript

// Exemplo da lógica central implementada (Clean Code)



- **Estruturas:** Foram utilizados `setInterval` para repetição e vetores para manipulação de logs.

4. Versionamento (Fase 5)

O projeto está organizado em um repositório Git com a seguinte estrutura:

- `/src`: Arquivos `.html`, `.css` e `.js`.
- `README.md`: Guia de instalação e objetivos.

CODIGOS PRONTO

HTML

```
1 <!DOCTYPE html>
2 <html Lang="pt-br">
3 <head>
4   <meta charset="UTF-8">
5   <title>Cruzamento 4.0 - Monitoramento CET</title>
6   <link rel="stylesheet" href="semaforo.css">
7 </head>
8 <body>
9   <div class="container">
10     <header>
11       <h1>🚦 Missão Cruzamento Seguro</h1>
12     </header>
13
14     <main>
15       <div class="cruzamento-layout">
16         <section class="semaforo-container">
17           <h3>Via Principal (A)</h3>
18           <div id="luz-vermelha-a" class="luz"></div>
19           <div id="luz-amarela-a" class="luz"></div>
20           <div id="luz-verde-a" class="luz"></div>
21         </section>
22
23         <section class="semaforo-container">
24           <h3>Via Secundária (B)</h3>
25           <div id="luz-vermelha-b" class="luz"></div>
26           <div id="luz-amarela-b" class="luz"></div>
27           <div id="luz-verde-b" class="luz"></div>
28         </section>
29       </div>
30
31       <section class="status-panel">
32         <h2>Status dos Sensores</h2>
33         <div class="card">Fluxo: <span id="val-fluxo">0%</span></div>
34         <div class="card">Clima: <span id="val-clima">Limp</span></div>
35         <div class="card">Sensor: <span id="val-sensor">Ativo</span></div>
36         <div class="card">Modo: <span id="val-modo">Normal</span></div>
37       </section>
38
39       <section class="botoes-container">
40         <h2>Simular Condições Especiais</h2>
41         <div class="botoes-grid">
42           <button id="btn-chuva" class="btn btn-chuva">☁️ Chuva</button>
43           <button id="btn-fluxo" class="btn btn-fluxo">🚗 Fluxo Alto</button>
44           <button id="btn-sensor" class="btn btn-sensor">⚠️ Falha Sensor</button>
45           <button id="btn-reset" class="btn btn-queda">🔥 Queda do Servidor</button>
46         </div>
47         <div class="botao-centralizado">
48           <button id="btn-resetar" class="btn btn-reset">🔄 Resetar Sistema</button>
49         </div>
50       </section>
51     </main>
52
53     <section class="logs">
54       <h3>Histórico do Sistema</h3>
55       <div id="log-list"></div>
56     </section>
57   </div>
58   <script src="semaforo.js"></script>
59 </body>
60 </html>
```

CSS

```
1 * {
2   margin: 0;
3   padding: 0;
4   box-sizing: border-box;
5 }
6
7 body {
8   font-family: 'Inter', sans-serif;
9   background: linear-gradient(135deg, #0f0f0f, #1c1c1c);
10  color: #f5f5f5;
11  min-height: 100vh;
12  display: flex;
13  justify-content: center;
14  align-items: center;
15 }
16
17 .container {
18   width: 95%;
19   max-width: 800px;
20   padding: 20px;
21 }
22
23 header {
24   text-align: center;
25   margin-bottom: 20px;
26   border-bottom: 1px solid #333;
27 }
28
29 .cruzamento-layout {
30   display: flex;
31   justify-content: space-around;
32   margin-bottom: 20px;
33 }
34
35 .semaforo-container {
36   background: #1b1b1b;
37   width: 100px;
38   padding: 20px;
39   border-radius: 20px;
40   border: 2px solid #505050;
41   display: flex;
42   flex-direction: column;
43   align-items: center;
44   gap: 20px;
45 }
46
47 .semaforo-container h3 {
48   font-size: 10px;
49   color: #ffffff;
50   text-transform: uppercase;
51 }
52
53 .luz {
54   width: 45px;
55   height: 45px;
56   border-radius: 50%;
57   background: #222;
58   transition: 0.3s;
59 }
60
```

```
61
62 .vermelho { background: #ff2e2e; box-shadow: 0 0 15px #ff2e2e; }
63 .amarelo { background: #ffcc00; box-shadow: 0 0 15px #ffcc00; }
64 .verde { background: #00d26a; box-shadow: 0 0 15px #00d26a; }
65
66 @keyframes piscar {
67   0%, 49% { opacity: 1; }
68   50%, 100% { opacity: 0.2; }
69 }
70
71 .amarelo-piscar {
72   animation: piscar 0.6s infinite;
73 }
74
75 .botoes-container {
76   background: #1b1b1b;
77   border: 2px solid #505050;
78   border-radius: 30px;
79   padding: 20px;
80   margin: 20px auto;
81   max-width: 550px;
82   width: calc(100% - 40px);
83   box-shadow: 0 4px 20px rgba(0,0,0,0.4);
84 }
85
86 .botoes-container h2 {
87   font-size: 14px;
88   margin-bottom: 15px;
89   text-align: center;
90 }
91
92 .botoes-grid {
93   display: grid;
94   grid-template-columns: repeat(2, 1fr);
95   gap: 10px;
96   justify-items: center;
97 }
98
99 .botao-centralizado {
100  display: flex;
101  justify-content: center;
102  margin-top: 10px;
103 }
104
105 .btn {
106   padding: 12px 30px;
107   border: none;
108   border-radius: 10px;
109   font-size: 13px;
110   font-weight: 100;
111   cursor: pointer;
112   transition: all 0.3s ease;
113   text-transform: uppercase;
114   box-shadow: 0 0 20px rgba(0, 0, 0, 0.5);
115 }
116
117 .btn:hover {
118   transform: translateY(-2px);
119   box-shadow: 0 0 20px rgba(0, 0, 0, 0.8);
120 }
121
122 .btn:active {
123   transform: translateY(0);
124 }
125
126 .btn-chuva {
127   background: #1ec7ff4d;
128   color: rgb(255, 255, 255);
129 }
130
131 .btn-chuva:hover {
132   background: #1ec7ff4d;
133 }
```

CSS

```
132     background: #1ec7ff4d;
133   }
134
135   .btn-fluxo {
136     background: #01471cfd;
137     color: white;
138   }
139
140   .btn-fluxo:hover {
141     background: #a5ffc79f;
142   }
143
144   .btn-sensor {
145     background: #d32f2f;
146     color: white;
147   }
148
149   .btn-sensor:hover {
150     background: #b71c1c;
151   }
152
153   .btn-queda {
154     background: #fcbbeb81;
155     color: white;
156   }
157
158   .btn-queda:hover {
159     background: #e9a3ff52;
160   }
161
162   .btn-reset {
163     background: #000000;
164     color: white;
165   }
166
167   .btn-reset:hover {
168     background: #333333;
169   }
170   .status-panel {
171     display: grid;
172     grid-template-columns: 1fr 1fr;
173     gap: 10px;
174     margin-top: 20px;
175   }
176
177   .card {
178     background: #222;
179     padding: 10px;
180     border-radius: 8px;
181     border-left: 4px solid #00d26a;
182   }
183
184   .logs {
185     background: #000;
186     margin-top: 20px;
187     padding: 15px;
188     height: 120px;
189     overflow-y: auto;
190     font-family: monospace;
191     color: #00ff00;
192     border-radius: 5px;
193   }
```


JS

```

1
2 const logHistorico = [];
3 let modoEspecial = null;
4 let falhaInterval = null;
5 let falhaColorIndex = 0;
6
7 function atualizarLogs(mensagem) {
8   const logList = document.getElementById('log-list');
9   const entrada = `[${new Date().toLocaleTimeString()}] ${mensagem}`;
10  logHistorico.push(entrada);
11
12  logList.innerHTML = logHistorico.slice(-4).reverse().join('<br>');
13 }
14
15 function gerenciarCruzamento() {
16   const status = {
17     fluxo: Math.floor(Math.random() * 100),
18     chuva: Math.random() > 0.85,
19     sensorAtivo: Math.random() > 0.1
20   };
21
22   // Não sobrescrever o modo especial (como falha descontrolada)
23   if (modoEspecial) return;
24
25   document.getElementById('val-fluxo').innerText = `${status.fluxo}%`;
26   document.getElementById('val-clima').innerText = status.chuva ? "Chuva Forte" : "Limpo";
27   document.getElementById('val-sensor').innerText = status.sensorAtivo ? "Ativo" : "FALHA";
28
29   const viaA = {
30     v: document.getElementById('luz-vermelha-a'),
31     a: document.getElementById('luz-amarela-a'),
32     g: document.getElementById('luz-verde-a')
33   };
34   const viaB = {
35     v: document.getElementById('luz-vermelha-b'),
36     a: document.getElementById('luz-amarela-b'),
37     g: document.getElementById('luz-verde-b')
38   };
39
40   const todasAsLuzes = [...Object.values(viaA), ...Object.values(viaB)];
41   todasAsLuzes.forEach(l => l.className = 'luz');
42
43   if (!status.sensorAtivo) {
44     viaA.a.classList.add('amarelo');
45     viaB.a.classList.add('amarelo');
46     document.getElementById('val-modo').innerText = "SEGURANÇA";
47     atualizarLogs("ALERTA: Sensor offline. Modo amarelo intermitente.");
48     return;
49   }
50
51   if (status.chuva) {
52     viaA.a.classList.add('amarelo');
53     viaB.v.classList.add('vermelho');

```

```

54     document.getElementById('val-modo').innerText = "CLIMA";
55     atualizarLogs("CAUTELA: Chuva detectada. Reduzindo velocidade.");
56   }
57   else if (status.fluxo > 70) {
58     viaA.g.classList.add('verde');
59     viaB.v.classList.add('vermelho');
60     document.getElementById('val-modo').innerText = "FLUXO ALTO";
61   }
62   else {
63     viaA.v.classList.add('vermelho');
64     viaB.g.classList.add('verde');
65     document.getElementById('val-modo').innerText = "NORMAL";
66     atualizarLogs("OPERAÇÃO: Tráfego normalizado.");
67   }
68 }
69
70 }
71
72 setInterval(gerenciarCruzamento, 3000);
73 gerenciarCruzamento();
74
75 function simularChuva() {
76   modoEspecial = 'chuva';
77   const viaA = {
78     v: document.getElementById('luz-vermelha-a'),
79     a: document.getElementById('luz-amarela-a'),
80     g: document.getElementById('luz-verde-a')
81   };
82   const viaB = {
83     v: document.getElementById('luz-vermelha-b'),
84     a: document.getElementById('luz-amarela-b'),
85     g: document.getElementById('luz-verde-b')
86   };
87
88   const todasAsLuzes = [...Object.values(viaA), ...Object.values(viaB)];
89   todasAsLuzes.forEach(l => l.className = 'luz');
90
91   viaA.a.classList.add('amarelo');
92   viaA.a.classList.add('amarelo-piscar');
93   viaB.v.classList.add('vermelho');
94   document.getElementById('val-modo').innerText = "CLIMA";
95   document.getElementById('val-clima').innerText = "Chuva Forte";
96   atualizarLogs("CAUTELA: Chuva detectada. Reduzindo velocidade.");
97 }
98
99 function simularFluxoAlto() {
100   modoEspecial = 'fluxo';
101   const viaA = {
102     v: document.getElementById('luz-vermelha-a'),
103     a: document.getElementById('luz-amarela-a'),
104     g: document.getElementById('luz-verde-a')
105   };
106   const viaB = {
107     v: document.getElementById('luz-vermelha-b'),
108     a: document.getElementById('luz-amarela-b'),
109     g: document.getElementById('luz-verde-b')
110   };
111
112   const todasAsLuzes = [...Object.values(viaA), ...Object.values(viaB)];
113   todasAsLuzes.forEach(l => l.className = 'luz');
114
115   viaA.g.classList.add('verde');
116   viaB.v.classList.add('vermelho');
117   document.getElementById('val-modo').innerText = "FLUXO ALTO";
118   document.getElementById('val-fluxo').innerText = "85%";
119   atualizarLogs("CONGESTIONAMENTO: Fluxo em 85%. Liberando Via A.");

```

JS

```

120 }
121
122 function simularFalhaSensor() {
123   modoEspecial = 'sensor';
124   const viaA = {
125     v: document.getElementById('luz-vermelha-a'),
126     a: document.getElementById('luz-amarela-a'),
127     g: document.getElementById('luz-verde-a')
128   };
129   const viaB = {
130     v: document.getElementById('luz-vermelha-b'),
131     a: document.getElementById('luz-amarela-b'),
132     g: document.getElementById('luz-verde-b')
133   };
134
135   const todasAsLuzes = [...Object.values(viaA), ...Object.values(viaB)];
136   todasAsLuzes.forEach(l => l.className = 'luz');
137
138   viaA.a.classList.add('amarelo');
139   viaA.a.classList.add('amarelo-piscar');
140   viaB.a.classList.add('amarelo');
141   viaB.a.classList.add('amarelo-piscar');
142   document.getElementById('val-modo').innerText = "SEGURANÇA";
143   document.getElementById('val-sensor').innerText = "FALHA";
144   atualizarLogs("ALERTA: Sensor offline. Modo amarelo intermitent
e.");
145 }
146
147 function Falhanoservidor() {
148   modoEspecial = 'falha';
149
150   if (falhaInterval) return;
151
152   const viaA = {
153     v: document.getElementById('luz-vermelha-a'),
154     a: document.getElementById('luz-amarela-a'),
155     g: document.getElementById('luz-verde-a')
156   };
157   const viaB = {
158     v: document.getElementById('luz-vermelha-b'),
159     a: document.getElementById('luz-amarela-b'),
160     g: document.getElementById('luz-verde-b')
161   };
162
163   const todasAsLuzes = [...Object.values(viaA), ...Object.values(viaB)];
164
165   document.getElementById('val-modo').innerText = "FALHA SERVIDOR";
166   document.getElementById('val-clima').innerText = "--";
167   document.getElementById('val-sensor').innerText = "--";
168   document.getElementById('val-fluxo').innerText = "--";
169   atualizarLogs("FALHA CRÍTICA: Servidor em modo descontrolado.");
170
171   const cores = ['vermelho', 'amarelo', 'verde'];
172   falhaColorIndex = 0;
173   todasAsLuzes.forEach(l => {
174     l.className = 'luz';
175     l.classList.add(cores[falhaColorIndex]);
176     if (cores[falhaColorIndex] === 'amarelo') l.classList.add('amarelo-piscar');
177   });
178   falhaInterval = setInterval(() => {
179     falhaColorIndex = (falhaColorIndex + 1) % cores.length;
180     const corAtual = cores[falhaColorIndex];

```

```

183   todasAsLuzes.forEach(l => {
184     l.className = 'luz';
185     l.classList.add(corAtual);
186     if (corAtual === 'amarelo') l.classList.add('amarelo-piscar');
187     else l.classList.remove('amarelo-piscar');
188   });
189   }, 1000);
190 }
191
192 function resetarSistema() {
193   modoEspecial = 'reset';
194
195   if (falhaInterval) {
196     clearInterval(falhaInterval);
197     falhaInterval = null;
198   }
199
200   const viaA = {
201     v: document.getElementById('luz-vermelha-a'),
202     a: document.getElementById('luz-amarela-a'),
203     g: document.getElementById('luz-verde-a')
204   };
205   const viaB = {
206     v: document.getElementById('luz-vermelha-b'),
207     a: document.getElementById('luz-amarela-b'),
208     g: document.getElementById('luz-verde-b')
209   };
210
211   const todasAsLuzes = [...Object.values(viaA), ...Object.values(viaB)];
212
213   const cores = ['verde', 'amarelo', 'vermelho'];
214   let idx = 0;
215
216   function aplicarCor(cor) {
217     todasAsLuzes.forEach(l => {
218       l.className = 'luz';
219       l.classList.add(cor);
220       if (cor === 'amarelo') l.classList.add('amarelo-piscar');
221       else l.classList.remove('amarelo-piscar');
222     });
223   }
224
225   document.getElementById('val-modo').innerText = "RESET";
226   document.getElementById('val-clima').innerText = "--";
227   document.getElementById('val-sensor').innerText = "--";
228   document.getElementById('val-fluxo').innerText = "--";
229   atualizarLogs("Sistema reiniciado com sucesso.");
230
231   aplicarCor(cores[0]);
232
233   setTimeout(() => {
234     aplicarCor(cores[1]);
235   }, 600);
236
237   setTimeout(() => {
238     aplicarCor(cores[2]);
239   }, 700);
240
241   setTimeout(() => {
242     modoEspecial = null;
243     document.getElementById('val-modo').innerText = "NORMAL";
244     document.getElementById('val-clima').innerText = "Livre";
245     document.getElementById('val-sensor').innerText = "Ativo";
246     document.getElementById('val-fluxo').innerText = "Normal";

```

```

246   document.getElementById('val-fluxo').innerText = "0%";
247   atualizarLogs("Sequência concluída. Sistema em modo normal.");
248   }, 600);
249
250   }, 700);
251   }, 800);
252 }
253
254 document.getElementById('btn-chuva').addEventListener('click', simularChuva);
255 document.getElementById('btn-fluxo').addEventListener('click', simularFluxoAlto);
256 document.getElementById('btn-sensor').addEventListener('click', simularFalhaSensor);
257 document.getElementById('btn-reset').addEventListener('click', Falhanoservidor);
258 document.getElementById('btn-resetar').addEventListener('click', resetarSistema);
259

```

PÁGINA WEB SEMÁFORO

VIA PRINCIPAL (A)

VIA SECUNDÁRIA (B)

Status dos Sensores

Clima: Limpo

Fluxo: 48%

Sensor: Ativo

Modo: NORMAL

Simular Condições Especiais

CHUVA

FLUXO ALTO

FALHA SENSOR

QUEDA DO SERVIDOR

RESETAR SISTEMA

Histórico do Sistema

[15:20:01] OPERAÇÃO: Tráfego normalizado.

[15:19:58] ALERTA: Sensor offline. Modo amarelo intermitente.

ORGANIZAÇÃO GITHUB

VERSIONAMENTO

Público

Assistir 0

Garfo 0

Estrela 0

principal

1 filial

0 Etiquetas

Acesse o arquivo

Adicionar arquivo

Código

Vivinzy

Adicionar arquivos por meio de upload

44a9d72 · 1 hora atrás

3 Commits

Fluxograma e Diagrama

Fluxograma e Diagrama

1 hora atrás

semáforo

Adicionar arquivos por meio de upload

1 hora atrás

README.md

Confirmação inicial

2 horas atrás

LEIA-ME

VERSIONAMENTO

Repositório completo do projeto

Sobre

Repositório completo do projeto

Leia-me

Atividade

0 estrelas

0 pessoas assistindo

0 garfos

Repositório de relatórios

Lançamentos

Nenhuma versão publicada

[Criar uma nova versão](#)

Pacotes

Nenhum pacote publicado.

principal

VERSIONAMENTO / Fluxograma e Diagrama

Acesse o arquivo

Adicionar arquivo

viviane019

Fluxograma e Diagrama

724a59 · 1 hora atrás

História

Nome	Última mensagem de commit	Data do último c...
...		
Diagrama.pdf	Fluxograma e Diagrama	1 hora atrás
Fluxograma.pdf	Fluxograma e Diagrama	1 hora atrás

principal

VERSIONAMENTO / semáforo

Acesse o arquivo

Adicionar arquivo

Vivinzy

Adicionar arquivos por meio de upload

44a9d72 · 1 hora atrás

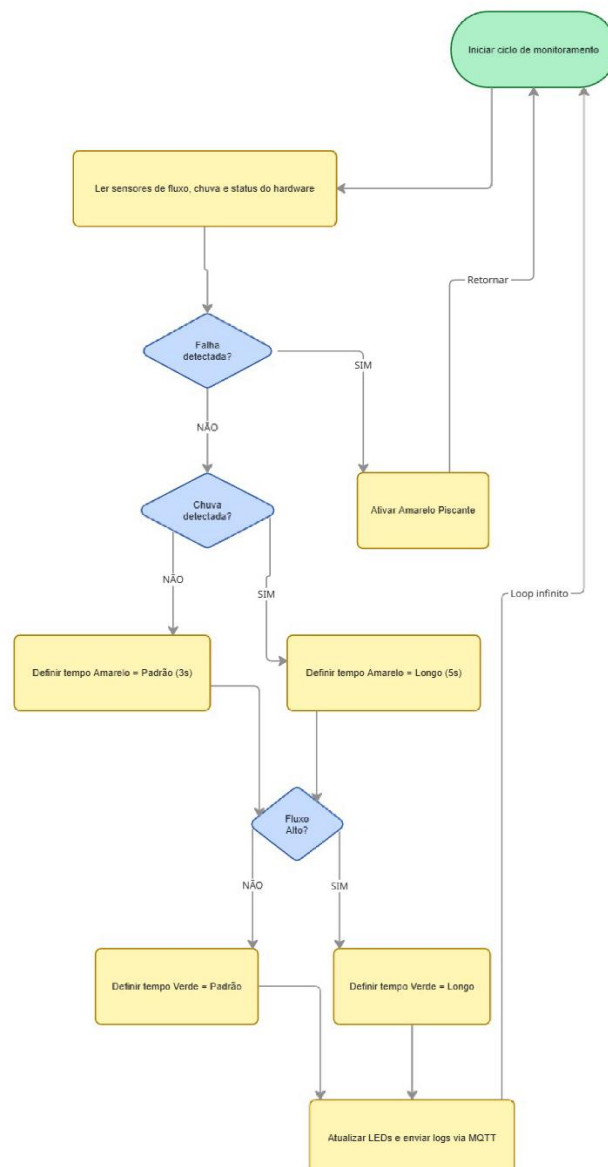
História

Nome	Última mensagem de commit	Data do último c...
...		
semaforo.css	Adicionar arquivos por meio de upload	1 hora atrás
semaforo.html	Adicionar arquivos por meio de upload	1 hora atrás
semaforo.js	Adicionar arquivos por meio de upload	1 hora atrás

FLUXOGRAMA E DIAGRAMA

Resumo do Fluxograma:

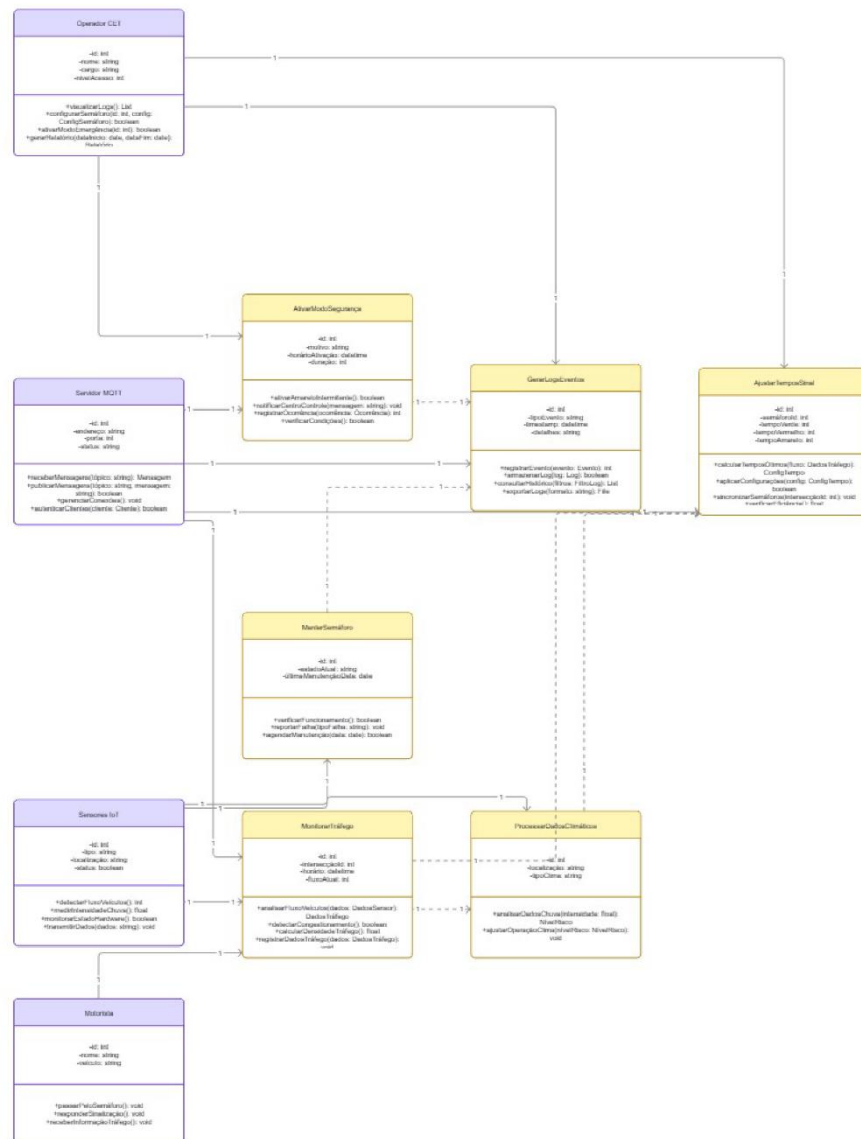
- Entrada: O sistema lê os sensores de carros e de chuva.
- Teste de Segurança: Se o sensor falhar, o sistema desvia para o modo "Amarelo Piscante".
- Processamento: O sistema decide se aumenta o tempo do Verde (muito carro) ou do Amarelo (chuva).
- Saída: Os LEDs físicos e a interface web são atualizados simultaneamente.
- Feedback: Os dados são enviados via MQTT para o servidor e o ciclo recomeça.



DIAGRAMA

Resumo do Diagrama:

- **Interação Central:** O sistema (controlador) recebe dados dos Sensores (fluxo e chuva) para decidir o tempo das luzes.
- **Ação do Motorista:** Ator que consome a sinalização visual gerada pelo sistema.
- **Gestão e Monitorização:** O Gestor da CET utiliza uma interface para controlo manual e o Servidor armazena os logs via MQTT.
- **Segurança Automática:** O diagrama mostra que, em caso de erro, o sistema assume sozinho o modo de segurança (amarelo piscante).



BIOGRAFIA:

<https://miro.com/pt/>